

Embedded Fire DetectorPi

IoT Curriculum

Giacomo Tortora
Andrea Bucci

0522501927
0522501969

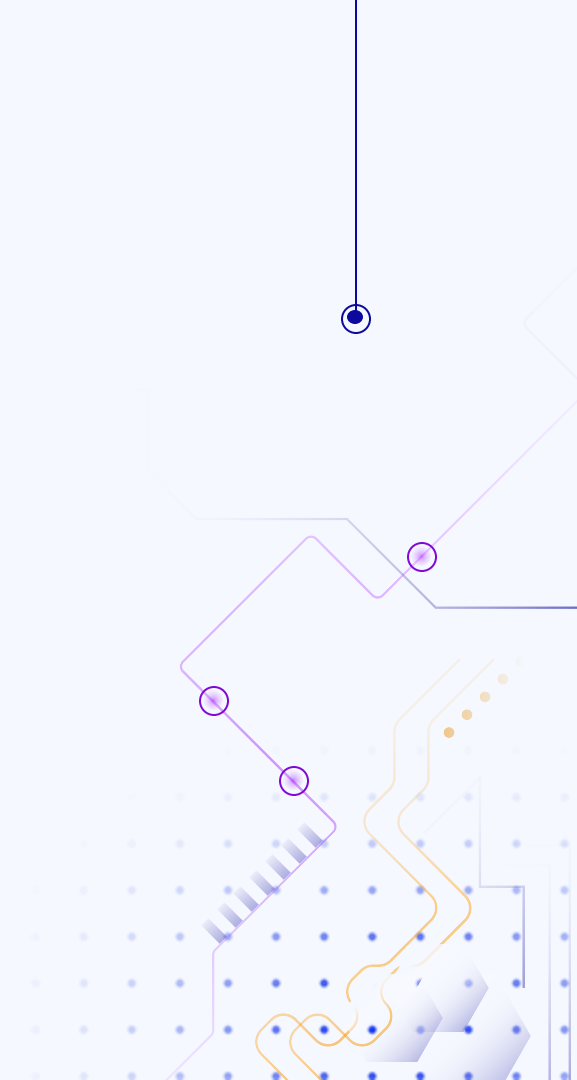




Table of contents

01

Project Overview

An overall overview of the project and its scope

03

Security

Security features applied to the project

02

HW and SW Implementation

Implementation of the project and technology used

04

Practical demonstration

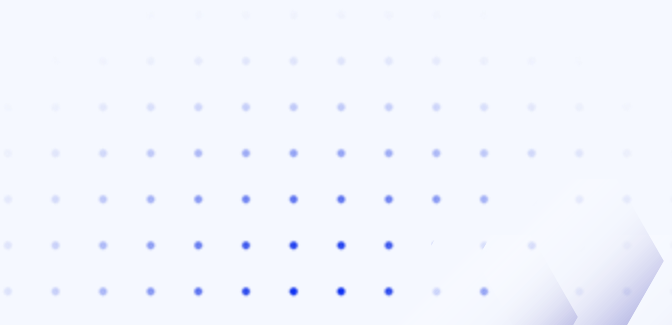
Real time application demonstration



01

Project Overview

An overall overview of the project and its scope



Scope of the Project

FireDetectorPi project aims to develop a **real-time fire detection system** running entirely **on-device on a Raspberry Pi**, leveraging a **machine learning model** for the recognition of flames and fire-related hazardous scenarios.

Overall, FireDetectorPi demonstrates the feasibility of **edge-based fire detection**, combining embedded systems and machine learning to support **early warning, safety, and prevention applications**.



Real-World Application

The **FireDetectorPi** system can be applied to a wide range of a **real-world fire prevention and early warning scenarios**, particularly where **continuous monitoring** and **fast local decision-making** are required.

Potential application include:

- **Residential environments** for early fire detection in homes and apartments
- **Industrial and warehouse facilities** to monitor high-risk areas and reduce response time
- **Educational and research settings** as a testbed for edge AI and embedded ML solutions



02

HW and SW

Implementation

Implementation of the project and technology used



Hardware Architecture

The hardware implementation of **FireDetectorPi** is based on a **RaspberryPi 3B+**, which act as the main embedded computing platform for the entire system.

All processing, including **machine learning inference** is executed **directly-on-device**, following an edge computing approach and avoiding any reliance on external cloud services.



Hardware Architecture

The ideal system is designed to acquire video input from a camera module. However, due to the **academic and educational nature** of this project and the unavailability of dedicated camera hardware, the current implementation uses **pre-recorded video files** to simulate live video streams.

This choice allows the validation of the detection pipeline while remaining independent of additional peripherals.



Software Architecture

The software architecture of **FireDetectorPi** is designed with a **modular and layered approach**, in order to clearly separate the main functional components of the system.

The application pipeline is composed of distinct stages, including **video input handling, frame preprocessing, ML inference** and **output management**.

ML model is fully integrated into the application and runs **locally on the RaspberryPi**, ensuring low latency and independence from network connectivity.

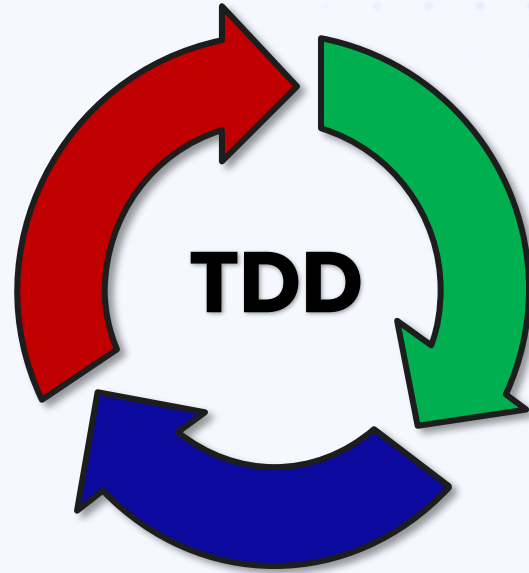
- **FrameProvider**: abstraction for PiCamera or video input.
- **ImageProcessor**: Pre-processing and input sanitization.
- **InferenceEngine**: Wrapper for the ML model.
- **FireMonitor**: Main orchestrator controlling the logic loop.
- **AlertNotifier**: Handles MQTT communication.

Software Architecture

The software implementation of **FireDetectorPi** has been developed using a **Test-Driven Development (TDD)** approach.

Tests were written before the actual code, ensuring that each module met its functional requirements from the start, following **RED-GREEN-REFACTORING** cycle.

This methodology improved **reliability and maintainability**, allowing early detection of errors and ensuring consistent behaviour across the entire pipeline.



Machine Learning Model & Inference

The core of the **FireDetectorPi** system is a **ML model** for fire and flame detection, designed to operate under the computational constraints of an embedded system. The model is trained using a **supervised learning approach** to recognize visual patterns associated with flames and fire-related scenarios.

To ensure feasibility on a **RaspberryPi**, the model is optimized for **on-device inference**, prioritizing a balance between detection accuracy, inference latency and resource consumption.

The model processes video data on a **frame-by-frame basis**, producing detection results in near real time. This approach allows continuous monitoring while maintaining stable performances on limited hardware.

The implementation demonstrated the practical applicability of **edge-based ML** for safety-critical tasks, while remaining scalable toward more powerful hardware and accelerated inference solutions in future deployments.

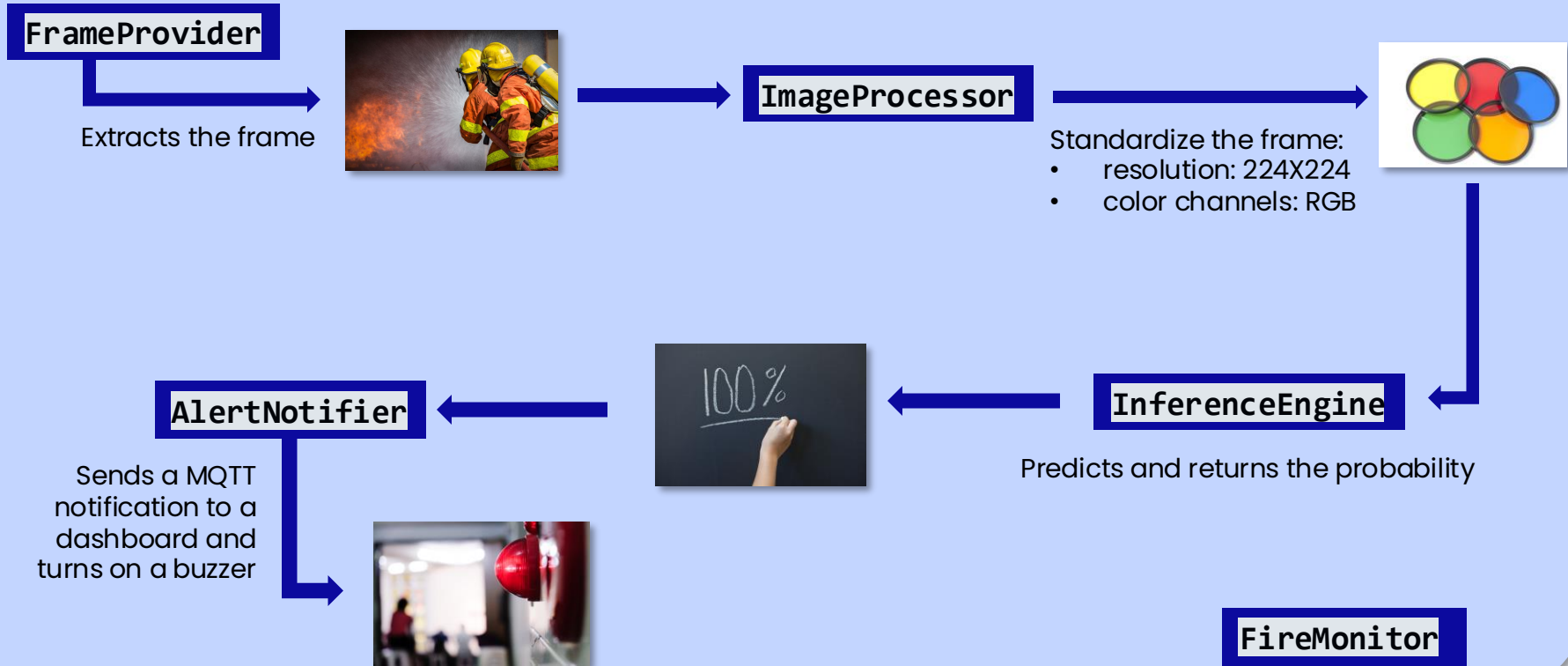
Machine Learning Model & Inference

The video processing pipeline of **FireDetectorPi** is designed to simulate real-time behaviour while operating within an academic and resource-constrained environment.

Video input is currently provided through pre-recorded video files. Each video stream is processed sequentially by extracting individual frames, which are then **preprocessed and analyzed by the ML model**.



Video Processing & System Workflow

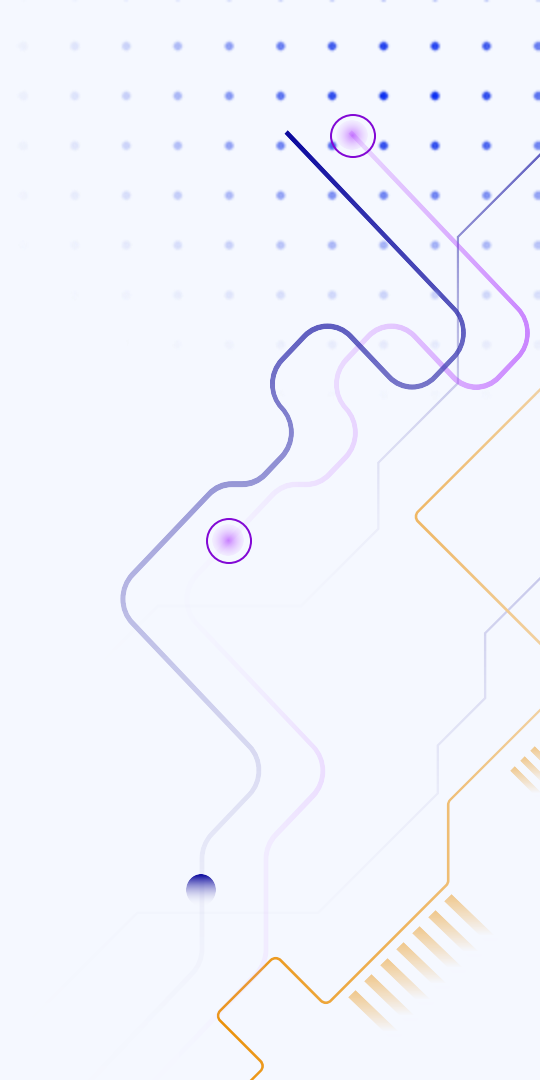
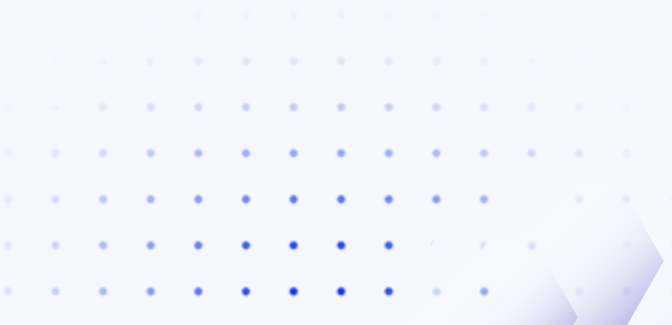




03

Security

Security features applied to the project



Security in ML - Evasion attacks

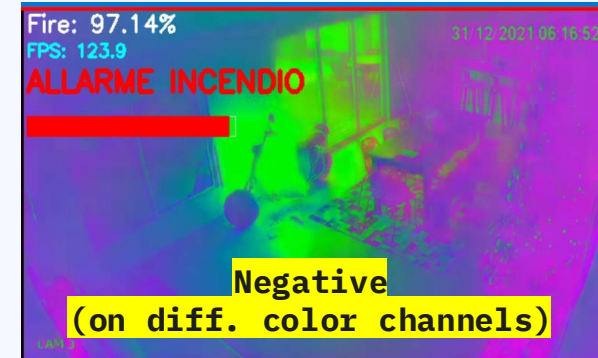
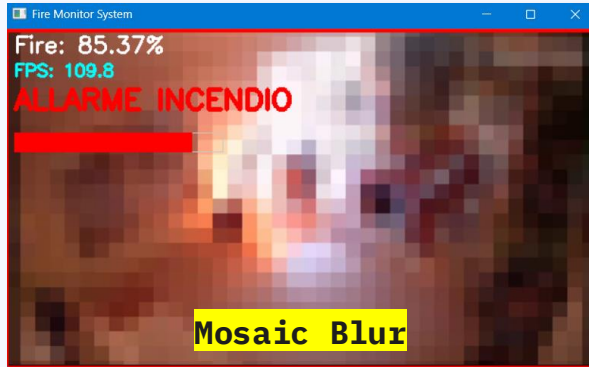
Security considerations in **FireDetectorPi** also involve the robustness of the machine learning model against **evasion attacks**. To mitigate this threat, security measures were incorporated **directly during the training phase** of the model.

Attacks consists of multiple video transformations like variations in color, saturation, brightness, lighting conditions, and image blur.

This strategy improves the model's ability to generalize and reduces sensitivity to adversarial or altered inputs, increasing resistance to evasion attempts aimed at bypassing fire detection.

By enhancing robustness at training time, the model becomes more resilient to environmental variations and intentional manipulations, improving reliability in real-world deployment scenarios.

Security in ML - Evasion attacks



System and Communication Security

In addition to model-level security, **FireDetectorPi** implements protection at the system communication level.

Detection events and system messages are transmitted to a monitoring dashboard using the **MQTT protocol**, commonly adopted in IoT environments.

To ensure **data confidentiality and integrity**, all MQTT messages are encrypted using **AES-256 encryption**.

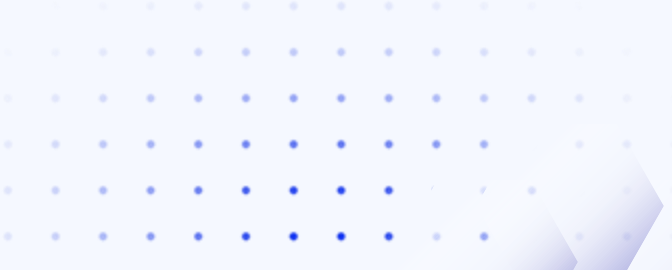
The combination of **secure communication channels** and **robust machine learning inference** contributes to a comprehensive security approach, addressing both **cybersecurity and adversarial ML threats** within the system.

```
INFO:root:!!! ALLARME FISICO AVVIATO !!!  
INFO:root:Dati criptati inviati:  
euwyysu298ma3iEIEUThlwZxTwlkKDIwrjwsXbY  
oAyybVLd5KWQRzC craS0x23gtln0jr/Nes7ldnzC  
utkXF57vcrQ+JMbGofP6oZkOHnajCXI7CCN7Qt  
eYz6dNXycph5dSz0Bv1m/YKuWZ4+PYw0A==
```

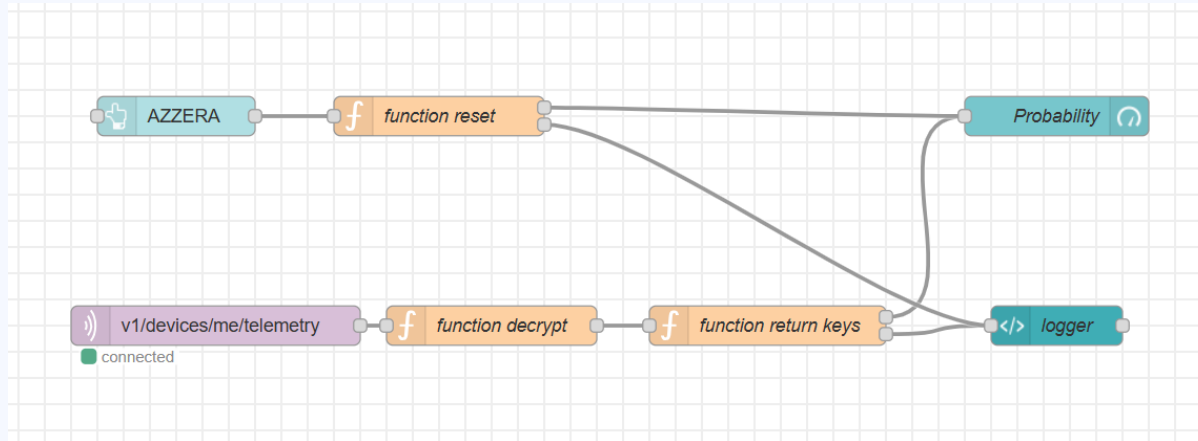


04 Practical demonstration

Real time application demonstration

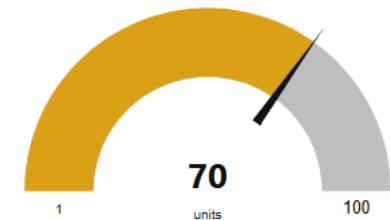


Node-Red dashboard



```
> [2026-01-24 18:53:57] STATO:  
FIRE_DETECTED - PROB: 70%  
> [2026-01-24 18:53:50] STATO:  
FIRE_DETECTED - PROB: 71%  
> [2026-01-24 18:53:41] STATO:  
FIRE_DETECTED - PROB: 73%  
> [2026-01-24 18:53:14] STATO:  
FIRE_DETECTED - PROB: 70%  
> [2026-01-24 18:53:03] STATO:  
FIRE_DETECTED - PROB: 70%  
> [2026-01-24 18:52:47] STATO:  
FIRE_DETECTED - PROB: 70%  
> [2026-01-24 18:52:39] STATO:  
FIRE_DETECTED - PROB: 71%
```

Probability





**Thanks for your
attention**

